Week 4 Agenda: Functions

1. Q&A
2. Nouns (data) vs. verbs (functions) in programming
3. Writing simple functions
4. Arguments and parameters
5. Return values
6. Default argument values
7. Documenting our functions
8. Special parmeters (e.g., `*args`)
9. Local vs. global variables

What are functions?

We've seen a bunch of functions (and methods) from the first day of this course:

- `print` (displays things on the screen)
- `input` (gets inputs from the user)
- `len` (returns the length of a string, list, tuple, or dict)
- `sum` (takes a list of numbers and returns their total)
- `type` (returns the type of value we hand it)

We don't need functions! Or we don't need more functions than Python comes with. We could make do with just the functions (and methods) that Python has, out of the box. And many of those are probably not absolutely necessary.

Why, then, do we have so many functions? Why are we going to talk today about writing new functions?

Abstraction. The idea of abstraction is that we can ignore the details and think at a higher level.

Writing functions allows us to go up a level (or more) of abstraction. We put the details in the function's definition, and once we know that the function works, we never need to think about those details again. Instead, we can just invoke the function, enjoy what it does, and ignore the internals. If and when we need to fix those internals, then we can open the function up -- but until then, we can think at our higher level.

Once I have a number of low-level functions defined, then I can work at a higher level. Much of programming is writing low-level functions so that we can think at a higher level, and never worrry about the low-level functions' internals again.

At the end of the day, a functions allows us to put a name to a bunch of functionality. It's basically defining a verb. A new verb that Python will know after we've defined it, and which will execute whatever steps we need.

So no... we don't need to be able to define functions... but it makes writing software *so* much easier and more straightforward.

Let's define a function!

To define a new function, we need a few things:

1. The keyword `def`, which means "define"
2. Immediately after `def`, we put the name of the function we want to define. This is really a variable assignment! The variable will be assigned a function value.
3. After the function name, we put `()`. Right now, these will be empty, but pretty soon we'll put parameter (variable) names inside of them.
4. At the end of the line, we put `:`. Why? Because the function body will start on the following line, and it will be indented.
5. The function body follows -- it can contain any code we want, at all. It's usually a good idea to keep the function body relatively short, but sometimes that isn't possible.

In [1]:
```
def hello():              # function definition line, with "def", empty () fo
    print('Hello!')       # one-line function body. Here, we're invoking "pri
```

Defining a function doesn't execute it. If you want to execute a function, you need to actually mention its name and then use `()`.

In [3]:
```
hello()   # When Python see (), it looks to its left and asks -- is this thi
```

```
Hello!
```

In [4]:
```
callable(hello)    # pass a function as an argument to another function!
```

Out[4]:
```
True
```

In [6]:
```
# what happens if I don't use ()?

hello    # we got back the function definition, or the function "object," bu
```

Out[6]:
```
<function __main__.hello()>
```

Exercise: Calculator

1. Define a function, `calc`, that when invoked asks the user three questions:

- What is the first number?
- What is the operator?
- What is the second number?

2. Assign the responses to `first`, `op`, and `second`, in order.
3. If `first` and `second` can be turned into integers, do so.
   - If not, then print an error message of some sort.
4. If the operator is either `+` or `-`, then calculate the result, and assign to `result`
   - If it's neither of these operators, then `result` can be set to `'bad operator'`
5. If the values are OK and the operator is OK, then print the entire expression plus its result.

Example:

```
calc()
First: 10
Operator: +
Second: 8
10 + 8 = 18
```

In [8]:
```
def calc():
    first = input('First: ').strip()
    op = input('Operator: ').strip()
    second = input('Second: ').strip()

    if first.isdecimal() and second.isdecimal():
        first = int(first)    # it's guaranteed that we can do this here, be
        second = int(second)

        if op == '+':
            result = first + second
        elif op == '-':
            result = first - second
        else:
            result = '(bad operator)'

        print(f'{first} {op} {second} = {result}')

    else:
        print(f'Both {first} and {second} need to be numeric; try again.')
```

In [9]:
```
calc()
```

```
2 + 8 = 10
```

In [10]:
```
calc()
```

```
15 - 8 = 7
```

In [11]:
```
calc()
```

```
Both hello and goodbye need to be numeric; try again.
```

In [12]:
```
calc()
```

```
2 * 5 = (bad operator)
```

This is annoying!

This function, `calc`, works -- but it requires that we be in front of our computer every time we want to run this function. It's very inconvenient for this to be the case.

It makes more sense for us to invoke the function, passing some values to it. Then we don't need to be pestered with the `input` function.

How would that look?

When we define our function, we put `()` after the name. We can, instead, put one or more variable names inside of the `()`. If we do that, then we're saying two things: (1) you must pass arguments to invoke this function, and (2) the arguments will be assigned to the parameters.

In [13]:
```
def hello(name):
    print(f'Hello, {name}!')
```

In [14]:
```
# parameters:  name
# arguments:  'Reuven'   # assignment of the first argument to the first par

hello('Reuven')
```

```
Hello, Reuven!
```

In [16]:
```
hello('someone else')
```

```
Hello, someone else!
```

In [17]:
```
# what happens if I now invoke hello without any arguments?

hello()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[17], line 3
      1 # what happens if I now invoke hello without any arguments?
      2
----> 3 hello()

TypeError: hello() missing 1 required positional argument: 'name'
```

In [18]:
```
hello('a', 'b')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[18], line 1
----> 1 hello('a', 'b')

TypeError: hello() takes 1 positional argument but 2 were given
```

When you're using `def` to define a function, you're assigning to a variable:

- The first `def hello` defined a function that take zero arguments.
- The second `def hello` defined a function that takes one argument.
- WHen we invoke `hello` after both definitions, only the second one is still around. The first is gone.

In [19]:
```
x = 5
x = 7

print(x)  # why doesn't it remember that x was one 5?
```

```
7
```

In [20]:
```
def hello(first_name, last_name):
    print(f'Hello, {first_name} {last_name}!')
```

In [21]:
```
hello()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 1
----> 1 hello()

TypeError: hello() missing 2 required positional arguments: 'first_name' and
'last_name'
```

In [22]:
```
hello('Reuven')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[22], line 1
----> 1 hello('Reuven')

TypeError: hello() missing 1 required positional argument: 'last_name'
```

In [23]:
```
hello('Reuven', 'Lerner')
```

```
Hello, Reuven Lerner!
```

Better `calc` (with parameters)

1. Modify the existing `calc` function (or take mine from GitHub, and copy):
   - It should take three arguments, each assigned to a different parameter (`first`, `op`, and `last`)

- It shouldn't use `input` to get values for its variables.
- Invoke `calc` a few times in a `for` loop with different values, and print the results.

In [28]:
```
def calc(first, op, second):
    if op == '+':
        result = first + second
    elif op == '-':
        result = first - second
    else:
        result = '(bad operator)'

    print(f'{first} {op} {second} = {result}')
```

In [30]:
```
# parameters: first    op    second
# arguments:    10    '+'      3

calc(10, '+', 3)
```

```
10 + 3 = 13
```

In [26]:
```
calc(15, '-', 2)
```

```
15 - 2 = 13
```

In [27]:
```
calc(20, '*', 6)
```

```
20 * 6 = (bad operator)
```

In [29]:
```
for i in range(10):
    calc(i, '+', i*3)
    calc(i, '-', i+2)
```

```
0 + 0 = 0
0 - 2 = -2
1 + 3 = 4
1 - 3 = -2
2 + 6 = 8
2 - 4 = -2
3 + 9 = 12
3 - 5 = -2
4 + 12 = 16
4 - 6 = -2
5 + 15 = 20
5 - 7 = -2
6 + 18 = 24
6 - 8 = -2
7 + 21 = 28
7 - 9 = -2
8 + 24 = 32
8 - 10 = -2
9 + 27 = 36
9 - 11 = -2
```

Next up

1. A little more about arguments and parameters
2. Return values from our functions

In [31]:
```
x = '123'      # we're assigning a string to x
x.isdecimal()  # this will return True or False -- we're invoking the string
```

Out[31]:
```
True
```

In [32]:
```
x = 123        # we're assigning an integer to x
x.isdecimal()  # this will result in an error -- because there is no integer
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[32], line 2
      1 x = 123        # we're assigning an integer to x
----> 2 x.isdecimal()  # this will result in an error -- because there is no
integer method int.isdecimal

AttributeError: 'int' object has no attribute 'isdecimal'
```

If our function assumes that it's going to get integer values, then you can't invoke `str.isdecimal` on them -- this is the whole point of methods, which are functions automatically assigned to a particular data type.

If your function will take strings, you can invoke `str.isdecimal` on the arguments. But if it takes integers, then you cannot.

How can you ensure that you get integers? This is a deep and difficult question that we'll address in a little bit.

In [35]:
```
def hello(name):
    print(f'Hello, {name}!')
```

In [36]:
```
# what kinds of values can I pass?
# answer: anything! Because in {} in an f-string, the value will be turned i

hello('world')
```

```
Hello, world!
```

In [37]:
```
hello(5)
```

```
Hello, 5!
```

In [38]:
```
hello([10, 20, 30])
```

```
Hello, [10, 20, 30]!
```

In [39]:
```
# the most extreme case!

hello(hello)   # pass the function to itself as an argument!
```

```
Hello, <function hello at 0x10a11bd70>!
```

In [40]:
```
# this can be a bit worrying, no?
# what if I want to ensure that only strings are passed?

# the real answer, for now, in Python, is.. switch to another programming la

# this is changing a bit -- we now have "type hints" in Python, but they're

def hello(name:str):
    print(f'Hello, {name}!')
```

In [41]:
```
hello('abcd')
```

```
Hello, abcd!
```

In [43]:
```
hello(5)   # the type hint does *NOTHING* here!
```

```
Hello, 5!
```

Type hints, aka "type annotations," are stored by Python, but not enforced.

They are enforced by (a) editors and (b) external programs that review your code, such as `mypy`.

Dynamic typing, which allows any variable to refer to any value, is very expressive and flexible. But yes, it opens the door to bugs.

In [45]:
```
# parameters:   first   second
# arguments:      10      3

def add(first, second):
    print(first + second)

add(10, 3)
```

```
13
```

In [46]:
```
# parameters:   first   second
# arguments:      100      200

def add(first, second):
    print(first + second)

add(100, 200)
```

```
300
```

In [47]:
```
# parameters:   first   second
# arguments:     'abcd'   'efgh'

def add(first, second):
    print(first + second)

add('abcd', 'efgh')
```

```
abcdefgh
```

Return values

When we invoke `input`, we can put it on the right side of assignment (`=`). That's only possible because it *returns* a value. That is, `input` gives us something back -- in mathematical terms, we can say that the call to `input` is replaced by a value.

`len` also returns a value, an integer.

What about other functions? What do they return? Each can return whatever it wants. `str.split`, for example, returns a list of strings.

And our functions, the ones we have defined so far today? They all return the special value `None`, because if you don't explicitly return a value in Python, the function returns `None`.

Some programming languages distinguish between function (which return values) and procedures (which don't). Python doesn't make this distinction, because *every* function returns something -- even if it's `None`.

In order to return a value, the function uses the keyword `return`:

- If there isn't any `return`, the function returns `None`
- If there is a `return`, but it doesn't have a value, the function returns `None`.
- Otherwise, whatever value is to the right of `return` is returned to the caller.

In [48]:
```
def hello(name):
    return f'Hello, {name}!'  # notice that return isn't a function -- so nc
```

In [49]:
```
s = hello('Reuven')   # notice -- the call to hello is on the right side of
```

In [50]:
```
print(f'>>> {s} <<<')
```

```
>>> Hello, Reuven! <<<
```

It's a far better idea for a function to return a value than to print it:

- If you `print` from your function, then the value cannot be retrieved/stored/used.
- If you `print` from your function, then the user doesn't have control over what is displayed and what is stored.

Exercise: `calc` with a return value

1. Use/steal/copy the `calc` from earlier, the one that takes arguments.
2. Modify `calc` further, such that it doesn't `print` the result on the screen, but returns a string value

3. Invoke `calc` in a `for` loop, capture the result string, and append it to a list.
4. Print the list of strings you created.

In [51]:
```
def calc(first, op, second):
    if op == '+':
        result = first + second
    elif op == '-':
        result = first - second
    else:
        result = '(bad operator)'

    return f'{first} {op} {second} = {result}'
```

In [55]:
```
calc(10, '-', 4)   # we'll see the quotes before/after the string, because w
```

Out[55]:
```
'10 - 4 = 6'
```

In Jupyter (and not in most normal Python systems!) if you return a value, then it is displayed. You have to `print` it to display it.

Normally, without `print`, things aren't displayed.

In [53]:
```
s = calc(10, '-', 4)
```

In [56]:
```
print(s)  # no quotes here, because we are printing for the end user
```

```
10 - 4 = 6
```

In [57]:
```
output = []

for i in range(10):
    s = calc(i+5, '-', i*3)   # calculate and assign to s
    output.append(s)          # append the string to output

print(output)
```

```
['5 - 0 = 5', '6 - 3 = 3', '7 - 6 = 1', '8 - 9 = -1', '9 - 12 = -3', '10 - 1
5 = -5', '11 - 18 = -7', '12 - 21 = -9', '13 - 24 = -11', '14 - 27 = -13']
```

You have now seen the outline of every function

1. Inputs come via arguments, and are assigned to parameters
2. Function body performs some calculations
3. Outputs are returned to the caller with `return`

What can we return to the caller?

In Python, the answer is simple: Absolutely any value we want.

- String
- Integer
- Float
- List
- Tuple
- Dict
- A combination of these

If you want to return a list of dicts, or a tuple of integers and strings, you absolutely can do that.

Can you return more than one value from a function?

No... but you can return a tuple containing more than one value. This is very common.

In [58]:
```
def hello_goodbye(name):
    return f'Hello, {name}!', f'Goodbye, {name}'   # a tuple is created wher
```

In [59]:
```
hello_goodbye('Reuven')
```

Out[59]:
```
('Hello, Reuven!', 'Goodbye, Reuven')
```

In [60]:
```
t = hello_goodbye('Reuven')
```

In [61]:
```
t
```

Out[61]:
```
('Hello, Reuven!', 'Goodbye, Reuven')
```

In [62]:
```
print(t[0])
```

```
Hello, Reuven!
```

In [63]:
```
print(t[1])
```

```
Goodbye, Reuven
```

In [64]:
```
# I can do even better with unpacking!

hello_greeting, goodbye_greeting = hello_goodbye('Reuven')
```

In [65]:
```
hello_greeting
```

Out[65]:
```
'Hello, Reuven!'
```

In [66]:
```
goodbye_greeting
```

Out[66]:
```
'Goodbye, Reuven'
```

Exercise: `multicalc`

1. Write a function, `multicalc`, that takes two arguments, both numbers.
2. The function should return a dict with 4 key-value pairs. The keys should be the strings for operators -- `+`, `-`, `*`, and `/`. The value will be the result of invoking that operator on the two numbers.

If I invoke

```
multicalc(10, 5)
```

I'll get

```
{'+':15, '-':5, '*':50, '/':2}
```

In [68]:
```
def multicalc(first, second):
    output = {'+': first + second,
              '-': first - second,
              '*': first * second,
              '/': first / second}
    return output

multicalc(10, 5)
```

Out[68]:
```
{'+': 15, '-': 5, '*': 50, '/': 2.0}
```

Next up

1. Docstrings and telling people how to invoke our functions
2. Complex return values
3. Keyword arguments

Docstrings

How can we tell people what a function does, and how to invoke it? We write a *docstring*.

A docstring is a string inside of the function -- it isn't assigned to a variable, and it isn't printed on the screen. It's just there, on the first line of the function.

When we define our function, Python notices the docstring and then stores it for when we want to know what a function does.

In [69]:
```
help(len)   # notice that I'm not invoking len, as len(), but I'm passing it
```

```
Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.
```

In [71]:
```
help(str.strip)
```

```
Help on method descriptor strip:

strip(self, chars=None, /) unbound builtins.str method
    Return a copy of the string with leading and trailing whitespace remove
d.

    If chars is given and not None, remove characters in chars instead.
```

In [72]:
```
help(hello)
```

```
Help on function hello in module __main__:

hello(name)
```

In [73]:
```
hello('a')
```

Out[73]:
```
'Hello, a!'
```

How to define a docstring

You can, in theory, use any string on the first line of a function body. But it's traditional to use triple-quoted strings -- that is, start with """ and end with """, which allows us to include newlines inside of the string.

The docstring shoudl then contain:

- First line is a short description
- Then can be a longer one
- We should also include three important clauses:
  - Expects: What arguments does the function expect to receive?
  - Modifies: What changes to the values and/or system does the function do?
  - Returns: What value(s) does the function return?

In [74]:
```
def hello(name):
    """Greet someone by name.

    This function follows a long tradition of getting a user's name and ther

    - Expects: One argument, a string (the user's name)
    - Modifies: Nothing
    - Returns: A string with the user's name and "Hello"
    """
```

```
    return f'Hello, {name}!'
```

In [75]:
```
hello('Reuven')
```

Out[75]:
```
'Hello, Reuven!'
```

In [76]:
```
help(hello)
```

```
Help on function hello in module __main__:

hello(name)
    Greet someone by name.

    This function follows a long tradition of getting a user's name and then
returning a friendly greeting that includes that name.

    - Expects: One argument, a string (the user's name)
    - Modifies: Nothing
    - Returns: A string with the user's name and "Hello"
```

Exercise: Document `calc`

Add a docstring to the most recent version of `calc`, which accepts arguments and returns a string.

In [79]:
```
def calc(first, op, second):
    """Perform a calculation and return a string with the result.

    Expects: three arguments:
        - first, an integer
        - op, a string that should be either '+' or '-'
        - second, another integer

        first and second can be floats, but we prefer integers
    Modifies: Nothing
    Returns:  A string with the arguments and their result.
    """

    if op == '+':
        result = first + second
    elif op == '-':
        result = first - second
    else:
        result = '(bad operator)'

    return f'{first} {op} {second} = {result}'


help(calc)
```

```
Help on function calc in module __main__:

calc(first, op, second)
    Perform a calculation and return a string with the result.

    Expects: three arguments:
        - first, an integer
        - op, a string that should be either '+' or '-'
        - second, another integer

        first and second can be floats, but we prefer integers
    Modifies: Nothing
    Returns:  A string with the arguments and their result.
```

Exercise: `analyze_word`

1. Define a function, `analyze_word`, which takes a string as an argument.
2. The function should return a tuple with three parts:
   - The original word (string)
   - The length of the word (integer)
   - Each character, and the number of times it appears in the word (as a dict)
3. Invoke the function, and use unpacking to retrieve each element.

In [80]:
```python
# how can we return multiple arguments?
# we just return one argument, a tuple, which has a few things in it

def myfunc():
    return (10, 20, 30)

myfunc()
```

Out[80]:
```
(10, 20, 30)
```

In [81]:
```python
# remember that we can define a tuple without () on the outside

def myfunc():
    return 10, 20, 30   # this is actually more typical

myfunc()
```

Out[81]:
```
(10, 20, 30)
```

In [82]:
```python
def analyze_word(text):
    character_counts = {}

    for one_character in text:
        if one_character in character_counts:      # if we've already seen t
            character_counts[one_character] += 1   # ... add 1 to its count
        else:
            character_counts[one_character] = 1    # if not, then add a new
```

```
    return text, len(text), character_counts
```

In [83]:
```
analyze_word('hello')
```

Out[83]:
```
('hello', 5, {'h': 1, 'e': 1, 'l': 2, 'o': 1})
```

Positional vs. keyword arguments

So far, we've seen that all arguments are *positional*. By this, I mean that the first argument is assigned to the first parameter, the second argument to the second parameter, etc.

But that can get annoying, especially if there are many parameters and you can get lost in them.

For this reason (and others), Python supports a second type of argument, the *keyword argument*.

Keyword arguments are always of the form `name=value`. They *must* have the name (of the parameter) specified. They must have the `=` before. You can pass them in any order you want, because the name is how they'll be assigned.

In [84]:
```
def sub(a, b):
    return a - b
```

In [87]:
```
# parameters:  a   b
# arguments:  10   3

sub(10, 3)
```

Out[87]:
```
7
```

In [88]:
```
# parameters:   a   b
# arguments:   10   3

sub(a=10, b=3)   # now we have passed both of our arguments as keyword argum
```

Out[88]:
```
7
```

In [89]:
```
# I can also say:

# parameters:   a   b
# arguments:   10   3


sub(b=3, a=10)
```

Out[89]:
```
7
```

In [90]:
```
# can we mix and match positional and keyword arguments?
# yes, but the rule is ALL POSITIONAL before ALL KEYWORD

sub(10, b=3)   # not a problem!
```

Out[90]: 7

In [91]:
```
sub(a=10, 3)
```

```
  Cell In[91], line 1
    sub(a=10, 3)
              ^
SyntaxError: positional argument follows keyword argument
```

Exercise: Count characters

1. Define a function, `count_characters`, that takes two arguments:
   - `text`, a string
   - `to_count`, another string
2. The function will return an integer -- the number of times tha one of the characters in `to_count` appears in `text`.
3. If we invoke the function with `to_count='aeiou'`, then it count the vowels in `text`. But if we pass it with `to_count='0123456789'`, then it will count the number of digits in `text`.
4. Once that works, invoke it three times:
   - Two positional arguments
   - Two keyword arguments
   - One of each

In [92]:
```
def count_characters(text, to_count):
    output = 0

    for one_character in text:           # go over each character in text
        if one_character in to_count:    # if the current character is in to
            output += 1                  # add 1 to output

    return output
```

In [93]:
```
count_characters('hello out there', 'aeiou')   # two positional
```

Out[93]: 6

In [94]:
```
count_characters(text='hello out there', to_count='aeiou')   # two keyword
```

Out[94]: 6

In [95]:
```
count_characters('hello out there', to_count='aeiou')   # one positional, or
```

Out[95]: 6

Next up

1. Default argument values
2. `*args`
3. Local vs. global variables

Default argument values

So far, in all of the functions that we have defined, the number of parameters and the number of arguments has been identical.

- One parameter, you need to pass one argument
- Two parameters, you need to pass two arguments

But we've already seen a few cases in which this isn't exactly true, either.

For example, if I want to split a string on `','` I invoke `str.split(',')`. But if I want to split it on whitespace, I can say `str.split()` -- without any argument.

How can this be?

We know that there is only one version of any given function. So what's going on?

The answer is: You can define a function with one or more parameters with default argument values. That is, if you invoke the function and pass an argument, then that argument is assigned to the parameter. But if you invoke the function and *don't* pass an argument, the default is used.

In [96]:
```
def sub(first, second):
    return first - second

sub(10, 3)
```

Out[96]: 7

In [97]:
```
sub(10)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[97], line 1
----> 1 sub(10)

TypeError: sub() missing 1 required positional argument: 'second'
```

In [98]:
```
def sub(first, second=1):   # here, we've given second a default argument va
    return first - second

sub(10, 3)
```

Out[98]: 7

In [99]:
```
sub(10)  # we only passed a value for first, which means that we'll use the
```

Out[99]: 9

In [100…
```
def sub(first=5, second=1):   # here, we'll give *both* parameters default v
    return first - second

sub(10, 3)    # pass both
```

Out[100… 7

In [101…
```
sub(10)       # pass first
```

Out[101… 9

In [102…
```
sub(second=4)   # now we're giving a value to second, but we aren't to first
```

Out[102… 1

Some rules for default argument values:

1. All mandatory parameters (i.e., without defaults) must come before optional parameters (i.e., with defaults).
2. It's a very bad idea to use mutable values as default arguments. So don't have defaults that are lists or dicts.
3. Python will retrieve the default for any parameter that it doesn't get a value for.

In [103…
```
help(str.split)   # what are the parameters for str.split?
```

```
Help on method descriptor split:

split(self, /, sep=None, maxsplit=-1) unbound builtins.str method
    Return a list of the substrings in the string, using sep as the separato
r string.

      sep
        The separator used to split the string.

        When set to None (the default value), will split on any
        whitespace character (including \n \r \t \f and spaces) and
        will discard empty strings from the result.
      maxsplit
        Maximum number of splits.
        -1 (the default value) means no limit.

    Splitting starts at the front of the string and works to the end.

    Note, str.split() is mainly useful for data that has been
    intentionally delimited.  With natural text that includes
    punctuation, consider using the regular expression module.
```

In [104…
```
s = 'abc de fgh ij klmn op qrst uvw x yz'
s.split()
```

Out[104…
```
['abc', 'de', 'fgh', 'ij', 'klmn', 'op', 'qrst', 'uvw', 'x', 'yz']
```

In [105…
```
s.split(maxsplit=3)
```

Out[105…
```
['abc', 'de', 'fgh', 'ij klmn op qrst uvw x yz']
```

Exercise: `count_characters` with two defaults

1. Rewrite `count_characters` from before such that `to_count` defaults to the vowels, `'aeiou'`, but can take any other string value.
2. Add another (third) parameter to the function, `lowercase`, which is a boolean (`True`/`False` value), defaulting to `False`. If it is `True`, then make the comparison case-insensitive by invoking `str.lower` on the string before looking for `to_count` characters.

In [106…
```
def count_characters(text, to_count='aeiou', lowercase=False):
    count = 0

    if lowercase:              # did we ask to compare in lowercase?
        text = text.lower()    # transform text, so that this will work

    for one_character in text:
        if one_character in to_count:
```

```
            count += 1

    return count
```

In [108…
```
count_characters('hello out there', 'abcdefghij')
```

Out[108…
```
5
```

In [109…
```
count_characters('hello out there', 'ABCDEFGHIJ')
```

Out[109…
```
0
```

In [111…
```
count_characters('HELLO OUT THERE', 'abcdefghij', True)
```

Out[111…
```
5
```

In [112…
```
# what if I want to check this string, and add lowercase to it, but I want t

count_characters('HELLO OUT THERE',  lowercase=True)   # use the default of
```

Out[112…
```
6
```

In [114…
```
# Tim wanted to know if we can get a string based on a range of characters
# we can use ord('c') or chr(n) to get a string back based on an integer --

chr(65)
```

Out[114…
```
'A'
```

In [115…
```
ord('h')
```

Out[115…
```
104
```

In [116…
```
def range_string(start, finish):
    output = []

    for index in range(start, finish):   # iterate over the range
        output.append(chr(index))        # get the character at index, and a

    return ''.join(output)               # return a string with the characte
```

In [117…
```
range_string(65, 75)
```

Out[117…
```
'ABCDEFGHIJ'
```

In [118…
```
def range_string_from_characters(start_char, end_char):
    return range_string(ord(start_char), ord(end_char))
```

In [119…
```
range_string_from_characters('j', 'x')
```

Out[119… 'jklmnopqrstuvw'

In [120…
```
count_characters('hello out there, everyone!',
                 range_string_from_characters('h', 'n'))
```

Out[120… 4

What about ... unknown numbers of arguments?

We've now seen that we can define a function takes a flexible number of arguments, and applies those to parameters, some of which have defaults.

But what if we don't know in advance how many arguments we're going to get? What if we want to accept any number of arguments?

That's where the special parameter `*args` comes into play.

If your function has a parameter named `*args` ("splat-args," because `*` looks like a bug that was squished), then:

- It must be the final parameter in your function
- Its value will be a tuple
- It will contain all of the positional arguments (not keyword arguments!) that were passed, and which no other parameter grabbed

In [121…
```
def mysum(numbers):
    total = 0

    for one_number in numbers:
        total += one_number

    return total
```

In [122…
```
mysum([10, 20, 30])
```

Out[122… 60

In [123…
```
# I'd like to pass the numbers *not* in a list, just as individual arguments

def mysum(*numbers):    # the * is mandatory, but the name doesn't have to b
    total = 0

    for one_number in numbers:
        total += one_number

    return total
```

In [124…
```
mysum(10, 20, 30)   # look, no list!
```

Out[124…
```
60
```

In [126…
```
def multchar(character, *numbers):
    for one_number in numbers:   # it is a tuple!
        print(character * one_number)
```

In [129…
```
multchar('*', 3, 5, 7, 1, 2, 6)
```

```
***
*****
*******
*
**
******
```

In [130…
```
print('a')
```

```
a
```

In [131…
```
print('b', 'c', 'd')
```

```
b c d
```

Exercise: `low_high`

1. Write a function, `low_high`, that takes any number of positional arguments, all integers.
2. We can assume there will be at least one argument.
3. The function returns a two-element list with the lowest and highest numbers.
4. (It's possible for them to be the same.)

In [132…
```
def low_high(*args):
    lowest = args[0]
    highest = args[0]

    for one_number in args[1:]:    # I iterate starting with args[1], since
        if one_number < lowest:
            lowest = one_number
        if one_number > highest:
            highest = one_number

    return [lowest, highest]
```

In [133…
```
low_high(10, 15, 2, 27, 50, 13)
```

Out[133…
```
[2, 50]
```

In [134…
```
1e5
```

Out[134… 100000.0

In [135…
```
1e7
```

Out[135… 10000000.0

In [136…
```
123_456  # this is treated as 123456
```

Out[136… 123456

Local vs. global variables

Let's say we have a function that uses a variable `x`.

And we have a separate function that uses a variable `x`.

We have a global variable, defined outside of those functions, called `x`.

How does Python make sense of all this? Are they all the same? Different? Overlapping?

The answer: There basically two types of variables in Python:

1. Global variables, meaning they're defined outside of any function.
2. Local variables, meaning that they're either parameters or they are assigned in the function body.

If you retrieve a variable value in a function, Python will first look for a local. If it cannot find one, then it raises an error.

Next time -- our final meeting for this course

1. Modules
2. Packages
3. Third-party packages and PyPI
4. `uv` and `pip`
5. Where do we go next?
6. AMA -- ask me anything

In [ ]:
```
```